PROJETO PRÁTICO · ACOMPANHAMENTO DO ESTUDANTE

Da ideia à solução com IA

Checklist de desenvolvimento com plataformas digitais e Vibecode / IA. Marque apenas o que já foi realizado e registre as evidências de cada etapa.

1. Definir o desafio → 2. Escolher a plataforma → 3. Integrar IA → 4. Testar e validar → 5. Documentar

Nomes dos estudantes (mínimo 2 e máximo 3)

Estudante 1 (obrigatório)

Lazim Francisco de Oliveira Júnior

Estudante 2 (obrigatório)

Lazim Francisco de Oliveira Junior

Estudante 3 (opcional)

Turma / disciplina

N1

Nome do projeto

Central de Chamados Técnicos

Data do preenchimento

29/10/2026

01

08/10

Definir o desafio

4 / 4

Escolha um problema simples e com escopo bem definido.

- ☑ Descrevi o problema e o público que será beneficiado.
- ☑ Defini o objetivo e o resultado esperado.
- ☑ Delimitei o que a solução fará e o que ficará fora do escopo.
- ☑ Estabeleci critérios para verificar se o desafio foi resolvido.

Problema, público, objetivo e critérios de sucesso

Problema: equipes pequenas recebem pedidos de suporte técnico por mensagem e e-mail, sem prazo definido, sem histórico e sem como saber quais problemas se repetem. Isso gera chamados esquecidos, atendimento sem prioridade e nenhuma métrica para melhorar.

Público: colaboradores que abrem pedidos de suporte (solicitantes), técnicos que atendem e um administrador que configura o sistema.

Objetivo e resultado esperado: um mini service desk web em que o solicitante abre e acompanha chamados, o técnico atende com prazos de SLA calculados automaticamente, e o administrador vê métricas de tempo de atendimento, cumprimento de SLA e reincidência, apoiado por uma base de conhecimento pesquisável.

Dentro do escopo: login com três perfis; chamados com categoria, prioridade e status; comentários públicos e notas internas; SLA por prioridade; dashboard; base de conhecimento; área administrativa; dados de demonstração sintéticos.

Fora do escopo: notificações por e-mail, anexos, recuperação de senha pelo app, integrações com outros sistemas, horário comercial e pausa de SLA, roteamento por especialidade.

Critérios de sucesso (cada um é verificável com teste):
C1. O solicitante abre um chamado e acompanha seu status.
C2. Os prazos de 1ª resposta e resolução são calculados pelo sistema a partir da prioridade.
C3. Um solicitante nunca vê chamado de outro e nunca vê notas internas.
C4. O técnico vê todos os chamados, filtra por SLA e atende.
C5. O dashboard mostra tempo médio, cumprimento de SLA e taxa de reincidência.
C6. A base de conhecimento retorna artigos por busca de palavra completa.
C7. Somente o administrador altera papéis, categorias e SLA.

02

15/10

Escolher e configurar a plataforma

4 / 4

Selecione a ferramenta adequada ao caso de uso: Bubble, Make, Zapier, Airtable, Pipefy, Adalo ou outra.

- ☑ Comparei as ferramentas considerando o caso de uso.
- ☑ Verifiquei limites de uso, custos e recursos necessários.
- ☑ Escolhi a plataforma e justifiquei minha decisão.
- ☑ Configurei o ambiente e executei um teste inicial.

Plataforma escolhida, justificativa e configuração realizada

Plataforma escolhida: Lovable (front-end React) com Lovable Cloud (banco Postgres, autenticação e controle de acesso por linha). O caso de uso exige login por perfil, isolamento de dados entre usuários, SLA calculado por regra e métricas agregadas. Isso pede um app com banco relacional, não apenas um fluxo de automação.

Comparação: Make e Zapier automatizam fluxos entre aplicativos, mas não entregam telas, login nem controle de acesso. Airtable cobre tabelas e formulários, com permissão por linha mais limitada e sem regras de SLA no banco. Pipefy já traz o conceito de chamado, mas é uma solução fechada e expõe pouco da lógica construída. Bubble atende o caso, com curva maior para regras de acesso e banco. Adalo é voltado a aplicativos mobile.

Limites e custos: o Lovable cobra por créditos de mensagem, e os créditos acabaram durante o projeto, o que limita novas correções por IA. A edição direta de tabelas e o uso do app continuam possíveis sem crédito. (Confira os valores atuais nos sites oficiais antes de citar números.)

Configuração e teste inicial: projeto criado e publicado, backend Lovable Cloud ativo, conta de usuário criada e confirmada, login realizado com sucesso.

03

22/10

Integrar Vibecode / IA

5 / 5

Dialogue com a IA de forma iterativa para estruturar configurações, lógicas leves ou integrações.

- ☑ Escrevi um prompt com objetivo, contexto e restrições.
- ☑ Revisei a resposta da IA e solicitei ajustes quando necessário.
- ☑ Implementei uma lógica, configuração ou integração compatível com o desafio.
- ☑ Registrei os prompts, as alterações e os resultados obtidos.
- ☑ Mantive senhas, chaves de API e dados pessoais fora dos prompts e do código público.

Prompts utilizados, integração implementada e ajustes realizados

Prompts utilizados: quatro prompts sequenciais, escritos com apoio do Claude e da skill prompt-master: (1) banco de dados, regras de acesso (RLS) e gatilhos; (2) autenticação e CRUD de chamados; (3) dashboard de métricas; (4) base de conhecimento, administração e dados de demonstração. Cada prompt define objetivo, contexto, proibições de escopo e critérios “Feito quando”.

Revisão e ajustes: a primeira ideia era um único prompt para o app inteiro. Foi descartada porque gera código acoplado e regras de acesso frágeis, então o banco foi feito primeiro, e as telas depois. Também revisei decisões de segurança: o cadastro nunca cria administrador (o papel é atribuído manualmente), e as métricas são calculadas no banco, não no navegador, pois o cliente é limitado a 1000 linhas por consulta.

Lógica implementada: cálculo de prazos de SLA por gatilho a partir da prioridade; marcação de recorrência; histórico de eventos do chamado; visão de SLA (no prazo, em risco, violado, cumprido); função de métricas do dashboard.

Segurança: nenhuma senha ou chave de API foi colocada em prompt ou código. Os dados de demonstração são sintéticos.

Prompt 1:
CONTEXTO
Projeto: "Central de Chamados" (mini service desk interno), app em português do Brasil. Stack fixa: React + Vite + TypeScript + Tailwind + shadcn/ui + Supabase (Auth + Postgres). Este passo cria SOMENTE o banco. NÃO crie telas, rotas ou componentes agora.

REGRAS FIXAS (valem para todos os passos)
- NUNCA criar edge functions, envio de e-mail, upload de arquivos, realtime, dark mode, i18n lib, features de IA ou qualquer tabela/coluna/rota/biblioteca que eu não tenha listado.
- NUNCA usar service role key no front. NUNCA usar localStorage/sessionStorage além do que o

supabase-js já usa.
- NUNCA usar dados mockados no código: tudo vem do banco.
- Segurança é feita por RLS e triggers. Checagem de papel no front é só UX.
- Não editar client.ts/types.ts gerados, exceto regeneração automática.
- Não me faça perguntas: onde houver ambiguidade, siga esta especificação à risca.
- Faça apenas o que foi pedido. Não refatore nem adicione extras.

TAREFA
Crie UMA migration com o SQL abaixo, exatamente como está (exceto o bloco SEED_KB, que você escreve). Se algum comando falhar, corrija o mínimo necessário, diga o que mudou, e NUNCA remova políticas RLS ou triggers. Pare e peça confirmação antes de qualquer DROP ou alteração fora deste SQL.

```
-- ===== TIPOS =====
create type public.app_role as enum ('admin','agent','requester');
create type public.ticket_status as enum ('open','in_progress','waiting_customer','resolved','closed');
create type public.ticket_priority as enum ('low','medium','high','urgent');
create type public.kb_status as enum ('draft','published');

-- ===== TABELAS =====
create table public.profiles (
  id uuid primary key references auth.users(id) on delete cascade,
  full_name text not null default '',
  email text,
  created_at timestamptz not null default now()
);

create table public.user_roles (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null unique references auth.users(id) on delete cascade,
  role public.app_role not null default 'requester'
);

create table public.categories (
  id uuid primary key default gen_random_uuid(),
  name text not null unique,
  active boolean not null default true
);

create table public.sla_policies (
  priority public.ticket_priority primary key,
  first_response_minutes int not null check (first_response_minutes > 0),
  resolution_minutes int not null check (resolution_minutes > 0)
```

```
);

create table public.tickets (
  id uuid primary key default gen_random_uuid(),
  number bigint generated always as identity unique,
  title text not null check (char_length(title) between 5 and 150),
  description text not null check (char_length(description) between 10 and 5000),
  category_id uuid not null references public.categories(id),
  priority public.ticket_priority not null default 'medium',
  status public.ticket_status not null default 'open',
  requester_id uuid not null references public.profiles(id),
  assignee_id uuid references public.profiles(id) on delete set null,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now(),
  first_response_at timestamptz,
  resolved_at timestamptz,
  closed_at timestamptz,
  first_response_due_at timestamptz,
  resolution_due_at timestamptz,
  recurrence_of_ticket_id uuid references public.tickets(id) on delete set null,
  reopen_count int not null default 0,
  is_demo boolean not null default false
);
create index tickets_requester_idx on public.tickets(requester_id);
create index tickets_status_idx on public.tickets(status);
create index tickets_assignee_idx on public.tickets(assignee_id);
create index tickets_created_idx on public.tickets(created_at desc);
create index tickets_recurrence_idx on public.tickets(requester_id, category_id, resolved_at);

create table public.ticket_comments (
  id uuid primary key default gen_random_uuid(),
  ticket_id uuid not null references public.tickets(id) on delete cascade,
  author_id uuid not null references public.profiles(id),
  body text not null check (char_length(body) between 1 and 5000),
  is_internal boolean not null default false,
  created_at timestamptz not null default now()
);
create index comments_ticket_idx on public.ticket_comments(ticket_id, created_at);

create table public.ticket_events (
  id uuid primary key default gen_random_uuid(),
  ticket_id uuid not null references public.tickets(id) on delete cascade,
  actor_id uuid references public.profiles(id) on delete set null,
```

```
  event_type text not null check (event_type in
('created','status_changed','priority_changed','assignee_changed')),
  from_value text,
  to_value text,
  created_at timestamptz not null default now()
);
create index events_ticket_idx on public.ticket_events(ticket_id, created_at);

create table public.kb_articles (
  id uuid primary key default gen_random_uuid(),
  title text not null check (char_length(title) between 5 and 150),
  body_md text not null check (char_length(body_md) >= 20),
  category_id uuid references public.categories(id) on delete set null,
  status public.kb_status not null default 'draft',
  author_id uuid references public.profiles(id) on delete set null,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now(),
  search_vector tsvector generated always as (
    setweight(to_tsvector('portuguese'::regconfig, coalesce(title,'')), 'A') ||
    setweight(to_tsvector('portuguese'::regconfig, coalesce(body_md,'')), 'B')
  ) stored
);
create index kb_search_idx on public.kb_articles using gin(search_vector);

-- ===== FUNÇÕES AUXILIARES =====
create or replace function public.has_role(_user_id uuid, _role public.app_role)
returns boolean language sql stable security definer set search_path = public as $$
  select exists (select 1 from public.user_roles where user_id = _user_id and role = _role)
$$;

create or replace function public.is_staff(_user_id uuid)
returns boolean language sql stable security definer set search_path = public as $$
  select exists (select 1 from public.user_roles where user_id = _user_id and role in ('admin','agent'))
$$;

create or replace function public.set_updated_at()
returns trigger language plpgsql as $$
begin new.updated_at := now(); return new; end $$;
create trigger kb_set_updated_at before update on public.kb_articles
  for each row execute function public.set_updated_at();

-- ===== NOVO USUÁRIO: perfil + papel requester (nunca admin automático) =====
create or replace function public.handle_new_user()
```

```
returns trigger language plpgsql security definer set search_path = public as $$
begin
  insert into public.profiles (id, full_name, email)
  values (new.id, coalesce(new.raw_user_meta_data->>'full_name',''), new.email);
  insert into public.user_roles (user_id, role) values (new.id, 'requester');
  return new;
end $$;
create trigger on_auth_user_created after insert on auth.users
  for each row execute function public.handle_new_user();

-- ===== TICKETS: BEFORE INSERT (SLA, recorrência, sanitização) =====
create or replace function public.tickets_before_insert()
returns trigger language plpgsql security definer set search_path = public as $$
declare v_sla public.sla_policies%rowtype; v_prev uuid;
begin
  if not public.is_staff(auth.uid()) then
    new.status := 'open'; new.assignee_id := null;
    new.first_response_at := null; new.resolved_at := null; new.closed_at := null;
    new.reopen_count := 0; new.is_demo := false;
    new.created_at := now(); new.updated_at := now();
  end if;
  select * into v_sla from public.sla_policies where priority = new.priority;
  if found then
    new.first_response_due_at := new.created_at + make_interval(mins =>
v_sla.first_response_minutes);
    new.resolution_due_at := new.created_at + make_interval(mins => v_sla.resolution_minutes);
  end if;
  -- Recorrência: mesmo solicitante + mesma categoria, chamado anterior resolvido nos 30 dias
antes da abertura deste
  select t.id into v_prev from public.tickets t
   where t.requester_id = new.requester_id and t.category_id = new.category_id
     and t.resolved_at is not null
     and t.resolved_at <= new.created_at
     and t.resolved_at >= new.created_at - interval '30 days'
   order by t.resolved_at desc limit 1;
  new.recurrence_of_ticket_id := v_prev;
  return new;
end $$;
create trigger tickets_bi before insert on public.tickets
  for each row execute function public.tickets_before_insert();

-- ===== TICKETS: BEFORE UPDATE (imutáveis, timestamps, reabertura, SLA por prioridade)
=====
```

```
create or replace function public.tickets_before_update()
returns trigger language plpgsql security definer set search_path = public as $$
declare v_sla public.sla_policies%rowtype;
begin
  new.id := old.id; new.requester_id := old.requester_id; new.created_at := old.created_at;
  new.recurrence_of_ticket_id := old.recurrence_of_ticket_id; new.is_demo := old.is_demo;
  new.updated_at := now();

  new.first_response_due_at := old.first_response_due_at;
  new.resolution_due_at := old.resolution_due_at;
  if new.priority is distinct from old.priority and old.status not in ('resolved','closed') then
    select * into v_sla from public.sla_policies where priority = new.priority;
    if found then
      new.first_response_due_at := new.created_at + make_interval(mins =>
v_sla.first_response_minutes);
      new.resolution_due_at := new.created_at + make_interval(mins => v_sla.resolution_minutes);
    end if;
  end if;

  if new.status is distinct from old.status then
    if new.status = 'resolved' then
      new.resolved_at := now();
    elsif new.status = 'closed' then
      new.closed_at := now();
      if new.resolved_at is null then new.resolved_at := now(); end if;
    elsif old.status in ('resolved','closed') and new.status in ('open','in_progress','waiting_customer')
then
      new.resolved_at := null; new.closed_at := null;
      new.reopen_count := old.reopen_count + 1;
    end if;
  end if;
  return new;
end $$;
create trigger tickets_bu before update on public.tickets
  for each row execute function public.tickets_before_update();

-- ===== TICKETS: auditoria =====
create or replace function public.tickets_log_events()
returns trigger language plpgsql security definer set search_path = public as $$
begin
  if tg_op = 'INSERT' then
    insert into public.ticket_events (ticket_id, actor_id, event_type, to_value)
    values (new.id, auth.uid(), 'created', new.status::text);
```

```
  else
    if new.status is distinct from old.status then
      insert into public.ticket_events (ticket_id, actor_id, event_type, from_value, to_value)
      values (new.id, auth.uid(), 'status_changed', old.status::text, new.status::text);
    end if;
    if new.priority is distinct from old.priority then
      insert into public.ticket_events (ticket_id, actor_id, event_type, from_value, to_value)
      values (new.id, auth.uid(), 'priority_changed', old.priority::text, new.priority::text);
    end if;
    if new.assignee_id is distinct from old.assignee_id then
      insert into public.ticket_events (ticket_id, actor_id, event_type, from_value, to_value)
      values (new.id, auth.uid(), 'assignee_changed', old.assignee_id::text, new.assignee_id::text);
    end if;
  end if;
  return new;
end $$;
create trigger tickets_ai after insert on public.tickets
  for each row execute function public.tickets_log_events();
create trigger tickets_au after update on public.tickets
  for each row execute function public.tickets_log_events();

-- ===== COMENTÁRIOS: 1ª resposta e retorno do solicitante =====
create or replace function public.comments_after_insert()
returns trigger language plpgsql security definer set search_path = public as $$
declare v_staff boolean; v_status public.ticket_status;
begin
  v_staff := public.is_staff(new.author_id);
  select status into v_status from public.tickets where id = new.ticket_id;
  if v_staff and not new.is_internal then
    update public.tickets set first_response_at = now()
     where id = new.ticket_id and first_response_at is null;
  end if;
  if not v_staff and v_status = 'waiting_customer' then
    update public.tickets set status = 'in_progress' where id = new.ticket_id;
  end if;
  return new;
end $$;
create trigger comments_ai after insert on public.ticket_comments
  for each row execute function public.comments_after_insert();

-- ===== REABERTURA PELO SOLICITANTE (único UPDATE permitido a ele) =====
create or replace function public.reopen_own_ticket(p_ticket_id uuid)
returns void language plpgsql security definer set search_path = public as $$
```

```
begin
  update public.tickets set status = 'open'
   where id = p_ticket_id and requester_id = auth.uid() and status = 'resolved';
  if not found then raise exception 'Chamado não encontrado ou não pode ser reaberto'; end if;
end $$;

-- ===== VIEW DE SLA (security_invoker: respeita RLS de quem consulta) =====
create view public.tickets_with_sla with (security_invoker = true) as
select t.*,
  case
    when t.first_response_at is not null then
      case when t.first_response_at <= t.first_response_due_at then 'met' else 'breached' end
    when now() > t.first_response_due_at then 'breached'
    when now() >= t.created_at + (t.first_response_due_at - t.created_at) * 0.8::float8 then 'at_risk'
    else 'on_track' end as first_response_sla,
  case
    when t.resolved_at is not null then
      case when t.resolved_at <= t.resolution_due_at then 'met' else 'breached' end
    when now() > t.resolution_due_at then 'breached'
    when now() >= t.created_at + (t.resolution_due_at - t.created_at) * 0.8::float8 then 'at_risk'
    else 'on_track' end as resolution_sla
from public.tickets t;

-- ===== RLS =====
alter table public.profiles enable row level security;
alter table public.user_roles enable row level security;
alter table public.categories enable row level security;
alter table public.sla_policies enable row level security;
alter table public.tickets enable row level security;
alter table public.ticket_comments enable row level security;
alter table public.ticket_events enable row level security;
alter table public.kb_articles enable row level security;

create policy profiles_select on public.profiles for select to authenticated
  using (id = auth.uid() or public.is_staff(auth.uid()));
create policy profiles_update on public.profiles for update to authenticated
  using (id = auth.uid()) with check (id = auth.uid());

create policy roles_select on public.user_roles for select to authenticated
  using (user_id = auth.uid() or public.is_staff(auth.uid()));
create policy roles_update on public.user_roles for update to authenticated
  using (public.has_role(auth.uid(),'admin') and user_id <> auth.uid())
  with check (public.has_role(auth.uid(),'admin') and user_id <> auth.uid());
```

```
create policy categories_select on public.categories for select to authenticated using (true);
create policy categories_insert on public.categories for insert to authenticated
  with check (public.has_role(auth.uid(),'admin'));
create policy categories_update on public.categories for update to authenticated
  using (public.has_role(auth.uid(),'admin')) with check (public.has_role(auth.uid(),'admin'));

create policy sla_select on public.sla_policies for select to authenticated
  using (public.is_staff(auth.uid()));
create policy sla_update on public.sla_policies for update to authenticated
  using (public.has_role(auth.uid(),'admin')) with check (public.has_role(auth.uid(),'admin'));

create policy tickets_select on public.tickets for select to authenticated
  using (requester_id = auth.uid() or public.is_staff(auth.uid()));
create policy tickets_insert on public.tickets for insert to authenticated
  with check (requester_id = auth.uid());
create policy tickets_update on public.tickets for update to authenticated
  using (public.is_staff(auth.uid())) with check (public.is_staff(auth.uid()));
create policy tickets_delete on public.tickets for delete to authenticated
  using (public.has_role(auth.uid(),'admin'));

create policy comments_select on public.ticket_comments for select to authenticated using (
  public.is_staff(auth.uid()) or (
    is_internal = false and exists (
      select 1 from public.tickets t
      where t.id = ticket_comments.ticket_id and t.requester_id = auth.uid())));
create policy comments_insert on public.ticket_comments for insert to authenticated with check (
  author_id = auth.uid() and (
    public.is_staff(auth.uid()) or (
      is_internal = false and exists (
        select 1 from public.tickets t
        where t.id = ticket_comments.ticket_id and t.requester_id = auth.uid() and t.status <>
'closed'))));

create policy events_select on public.ticket_events for select to authenticated
  using (public.is_staff(auth.uid()));

create policy kb_select on public.kb_articles for select to authenticated
  using (status = 'published' or public.is_staff(auth.uid()));
create policy kb_insert on public.kb_articles for insert to authenticated
  with check (public.is_staff(auth.uid()));
create policy kb_update on public.kb_articles for update to authenticated
  using (public.is_staff(auth.uid())) with check (public.is_staff(auth.uid()));
```

```
create policy kb_delete on public.kb_articles for delete to authenticated
  using (public.is_staff(auth.uid()));

-- ===== SEED =====
insert into public.categories (name) values
  ('Acesso e Senhas'),('Hardware'),('Software'),('Rede e Internet'),('E-mail'),('Outros');

insert into public.sla_policies (priority, first_response_minutes, resolution_minutes) values
  ('urgent', 30, 240), ('high', 60, 480), ('medium', 240, 1440), ('low', 480, 4320);

-- SEED_KB (você escreve): insira 6 artigos com status 'published', author_id null, em português, markdown,
-- 60 a 120 palavras cada, passos numerados realistas e genéricos, sem nomes de empresas ou pessoas reais.
-- Temas: redefinir senha; configurar VPN; impressora offline; Wi-Fi instável; e-mail não sincroniza; computador lento.
-- Associe cada um à categoria correspondente pelo name.
```

FEITO QUANDO
- A migration aplica sem erro; as 8 tabelas e a view existem com RLS habilitada nas 8 tabelas.
- Nenhum arquivo de front foi criado ou alterado.
- Ao final, liste em até 10 linhas qualquer desvio que você precisou fazer em relação ao SQL.

Prompt 2:

CONTEXTO
Banco do Passo 1 já existe (tabelas, RLS, triggers, view tickets_with_sla, função reopen_own_ticket). Agora construa o front de autenticação e chamados. UI 100% em pt-BR.

REGRAS FIXAS
- NUNCA criar edge functions, e-mail, upload, realtime, dark mode, IA, ou bibliotecas além de: react-router-dom, @tanstack/react-query, react-hook-form, zod, @hookform/resolvers, date-fns, shadcn/ui, lucide-react.
- NUNCA mock de dados. NUNCA localStorage próprio. NUNCA service role no front.
- Segurança vem do RLS; guardas de rota são só UX.
- Não altere SQL/migrations neste passo. Se achar que falta algo no banco, PARE e me avise.
- Sem perguntas; siga a especificação. Faça somente o pedido.

DESIGN
Sóbrio, produtividade tipo SaaS: fundo branco/cinza-claro, cor de destaque única azul, tipografia Inter, cards com borda fina, sem gradientes, sem ilustrações. Sidebar fixa em desktop; em <768px vira Sheet com botão de menu. Estados de loading (Skeleton), vazio (mensagem + ação) e erro (toast com mensagem) em toda lista e formulário.

Cores semânticas: prioridade urgent=vermelho, high=laranja, medium=azul, low=cinza. Status: open=azul, in_progress=roxo, waiting_customer=âmbar, resolved=verde, closed=cinza. SLA: met=verde, on_track=cinza, at_risk=âmbar, breached=vermelho. Rótulos pt-BR: Aberto, Em andamento, Aguardando solicitante, Resolvido, Fechado; Baixa, Média, Alta, Urgente; Cumprido, No prazo, Em risco, Violado.

AUTH
- Provider único (AuthProvider): configure onAuthStateChange ANTES de getSession; exponha session, user, role e loading.
- role = user_roles.role onde user_id = auth.uid() (maybeSingle, via React Query). Se vier nulo, trate como erro de conta e mostre tela "Conta sem perfil, contate o administrador" com botão Sair.
- /login: abas Entrar e Criar conta (nome completo, e-mail, senha mínimo 8). Cadastro usa signUp com options.data.full_name e emailRedirectTo = window.location.origin. Mensagens de erro do Supabase traduzidas para pt-BR para: credenciais inválidas, e-mail já cadastrado, senha curta.
- Rota "/" redireciona: staff (admin/agent) → /tickets; requester → /tickets. (Dashboard entra no Passo 3.)
- ProtectedRoute: sem sessão → /login. Aguarde loading antes de redirecionar (sem flash).

LAYOUT
Sidebar com: "Chamados" (/tickets), "Novo chamado" (/tickets/new). Rodapé: nome do usuário, rótulo do papel (Solicitante/Agente/Administrador) e botão Sair. NÃO crie links para páginas que ainda não existem.

LISTA /tickets
- Requester vê "Meus chamados"; staff vê "Chamados". Query em tickets_with_sla (RLS filtra sozinho).
- Colunas: #número, título, categoria, prioridade, status, SLA de resolução (SOMENTE staff), responsável (SOMENTE staff), criado em (dd/MM/yyyy HH:mm).
- Nomes de categoria e de responsável: NÃO faça embed (view não tem relacionamento). Busque categories (e, para staff, profiles) em queries separadas e una em memória.
- Paginação no servidor: 20 por página com range() e count exact. Ordenação fixa por created_at desc.
- Filtros no servidor: status, prioridade, categoria, busca por título (ilike). Staff também: responsável (Todos / Meus / Sem responsável / agente específico) e SLA (Violado / Em risco). Filtros refletidos na URL (search params).
- Linha clicável → /tickets/:id.

NOVO CHAMADO /tickets/new
- Campos: título (5–150), descrição (10–5000), categoria (somente ativas), prioridade (padrão Média). Validação zod com exatamente esses limites.
- Insert em tickets com requester_id = user.id; NÃO envie status, due dates nem assignee (o trigger cuida). Após sucesso, navegue para /tickets/:id. Bloqueie duplo clique no botão.

DETALHE /tickets/:id
- Busca em tickets_with_sla por id (maybeSingle). Se vier nulo: "Chamado não encontrado" (não distinga inexistente de sem acesso).
- Cabeçalho: #número, título, badges de status/prioridade, categoria, criado em, e se recorrente: badge "Recorrente" com link para o chamado anterior (recurrence_of_ticket_id) — para staff.
- Corpo: descrição, depois thread de comentários em ordem cronológica.
- Autor dos comentários: staff vê nome via profiles. Requester NÃO consegue ler perfis de terceiros: para comentários de outros autores mostre "Equipe de suporte"; para os dele, "Você". Comentários internos (is_internal) têm destaque âmbar e rótulo "Nota interna" (só staff os recebe do banco).
- Formulário de comentário (1–5000): requester envia is_internal=false sempre, e o formulário some se status = closed. Staff tem switch "Nota interna" (padrão desligado).
- Requester com status resolved: botão "Reabrir chamado" que chama supabase.rpc('reopen_own_ticket', { p_ticket_id }).
- Painel lateral SOMENTE staff: selects de status (todos os 5), prioridade, responsável (lista = user_roles com role admin/agent + nomes de profiles; opção "Sem responsável"); botão "Assumir chamado" (assignee = eu e, se status open, status = in_progress); SLA com dois badges: "1ª resposta" e "Resolução", cada um com tooltip da data limite; contador de reaberturas; timeline de ticket_events (ordem cronológica; texto ex.: "Status: Aberto → Em andamento", "Responsável alterado") com nome do ator.
- Toda mutação: toast de sucesso/erro e invalidar queries do chamado, comentários, eventos e lista.
- Nunca calcule SLA, datas limite ou timestamps no cliente: use somente o que o banco devolve.

FEITO QUANDO
- Cadastro e login funcionam; sair limpa a sessão.
- Requester só lista os próprios chamados; staff lista todos com filtros e paginação.
- Criar chamado preenche due dates e status 'open' sem o front enviar esses campos.
- Comentário interno não aparece para requester (garantido por RLS, não por filtro no front).
- Nenhuma página fora de: /login, /tickets, /tickets/new, /tickets/:id.
- Ao final, liste em até 8 linhas o que NÃO foi implementado ou ficou diferente do pedido.

Prompt 3:

CONTEXTO
Passos 1 e 2 prontos. Agora: dashboard de métricas para staff, com agregação feita NO BANCO (o Supabase limita 1000 linhas por query; agregar no cliente dá números errados).

REGRAS FIXAS
- NUNCA criar edge functions, e-mail, upload, realtime, dark mode, IA. Única biblioteca nova permitida: recharts.
- NUNCA mock de dados nem agregar tickets no cliente. NUNCA localStorage.
- Faça somente o pedido. Sem perguntas.
- Se o SQL falhar, corrija o mínimo, diga o que mudou, e NUNCA remova a checagem is_staff.

PARTE A — nova migration (exatamente como está):

```
create or replace function public.dashboard_metrics(p_days int default 30)
returns jsonb language plpgsql stable security invoker set search_path = public as $$
declare
  v_from timestamptz;
  v_today date := (now() at time zone 'America/Sao_Paulo')::date;
  v_result jsonb;
begin
  if not public.is_staff(auth.uid()) then raise exception 'Acesso negado'; end if;
  if p_days < 1 or p_days > 365 then raise exception 'p_days inválido'; end if;
  v_from := ((v_today - (p_days - 1))::timestamp) at time zone 'America/Sao_Paulo';

  select jsonb_build_object(
    'open_count', (select count(*) from public.tickets where status = 'open'),
    'in_progress_count', (select count(*) from public.tickets where status = 'in_progress'),
    'waiting_customer_count', (select count(*) from public.tickets where status = 'waiting_customer'),
    'breached_active_count', (select count(*) from public.tickets_with_sla
       where status in ('open','in_progress','waiting_customer')
         and (first_response_sla = 'breached' or resolution_sla = 'breached')),
    'created_in_period', (select count(*) from public.tickets where created_at >= v_from),
    'resolved_in_period', (select count(*) from public.tickets where resolved_at >= v_from),
    'avg_first_response_minutes', (select round((avg(extract(epoch from (first_response_at -
created_at))) / 60)::numeric, 1)
       from public.tickets where created_at >= v_from and first_response_at is not null),
    'avg_resolution_minutes', (select round((avg(extract(epoch from (resolved_at - created_at))) /
60)::numeric, 1)
       from public.tickets where resolved_at >= v_from),
    'resolution_sla_compliance_pct', (select round(100.0 * count(*) filter (where resolved_at <=
resolution_due_at) / nullif(count(*), 0), 1)
       from public.tickets where resolved_at >= v_from),
    'recurrence_rate_pct', (select round(100.0 * count(*) filter (where recurrence_of_ticket_id is not
null) / nullif(count(*), 0), 1)
       from public.tickets where created_at >= v_from),
    'by_category', (select coalesce(jsonb_agg(to_jsonb(x) order by x.total desc), '[]'::jsonb) from (
       select c.name, count(*) as total,
              count(*) filter (where t.recurrence_of_ticket_id is not null) as recurrent
       from public.tickets t join public.categories c on c.id = t.category_id
       where t.created_at >= v_from group by c.name) x),
    'by_day', (select coalesce(jsonb_agg(to_jsonb(y) order by y.day), '[]'::jsonb) from (
       select g.d::date as day,
         (select count(*) from public.tickets t where (t.created_at at time zone
'America/Sao_Paulo')::date = g.d::date) as created,
```

```
        (select count(*) from public.tickets t where (t.resolved_at at time zone
'America/Sao_Paulo')::date = g.d::date) as resolved
       from generate_series(v_today - (p_days - 1), v_today, interval '1 day') as g(d)) y)
  ) into v_result;
  return v_result;
end $$;
```

PARTE B — front: rota /dashboard (somente staff)
- Guarda de rota: requester que acessar /dashboard é redirecionado para /tickets. Staff: "/" passa a redirecionar para /dashboard. Adicione "Dashboard" no topo da sidebar SOMENTE para staff.
- Seletor de período: 7, 30 (padrão) e 90 dias. Chamada: supabase.rpc('dashboard_metrics', { p_days }). Defina uma interface TypeScript manual para o retorno (os tipos gerados podem não incluir a função) e faça cast.
- Linha 1 (snapshot atual, NÃO dependem do período): Abertos, Em andamento, Aguardando solicitante, Ativos com SLA violado (destaque vermelho se > 0).
- Linha 2 (período): Criados, Resolvidos, Tempo médio de 1ª resposta, Tempo médio de resolução, % SLA de resolução cumprido, Taxa de reincidência. Valor nulo → "—". Minutos formatados: <60 → "42 min"; ≥60 → "3h 20min"; ≥1440 → "2d 4h".
- Gráfico 1 (recharts LineChart): criados vs resolvidos por dia (by_day), eixo X dd/MM, tooltip em pt-BR.
- Gráfico 2 (recharts BarChart horizontal): chamados por categoria com total e recorrentes em barras separadas (by_category).
- Abaixo dos gráficos, texto pequeno e fixo: "Reincidência = chamado aberto pelo mesmo solicitante, na mesma categoria, até 30 dias após ele ter um chamado resolvido. É uma heurística por categoria, não identifica causa raiz."
- Skeleton enquanto carrega; erro com toast + botão "Tentar novamente"; se created_in_period = 0, estado vazio "Sem chamados no período".

FEITO QUANDO
- /dashboard bloqueado para requester e funcional para staff.
- Nenhum cálculo de métrica no cliente além de formatação.
- Troca de período refaz a chamada RPC e atualiza tudo.
- Liste em até 6 linhas qualquer desvio.

Prompt 4:
CONTEXTO
Passos 1–3 prontos. Agora: base de conhecimento (KB), sugestão de artigos ao abrir chamado, área administrativa e gerador de dados sintéticos.

REGRAS FIXAS
- NUNCA criar edge functions, e-mail, upload, realtime, dark mode, IA. Única biblioteca nova permitida: react-markdown (SEM plugin de HTML bruto, para evitar XSS).
- NUNCA mock de dados no código. NUNCA localStorage. Faça somente o pedido. Sem perguntas.

- Se o SQL falhar, corrija o mínimo e diga o que mudou; NUNCA remova checagens has_role/is_staff.

PARTE A — nova migration (exatamente como está):

```
create or replace function public.seed_demo_tickets(p_count int default 60)
returns int language plpgsql security definer set search_path = public as $$
declare
  v_req uuid[]; v_staff uuid[]; v_cat uuid[];
  v_titles text[] := array['Não consigo acessar minha conta','Computador muito lento','Impressora não imprime',
    'Sem acesso à VPN','Erro ao abrir planilha','E-mail não sincroniza','Wi-Fi instável',
    'Solicitação de novo software','Tela azul ao iniciar','Redefinição de senha'];
  v_prios public.ticket_priority[] := array['low','medium','medium','high','urgent']::public.ticket_priority[];
  i int; v_created timestamptz; v_prio public.ticket_priority; v_sla public.sla_policies%rowtype;
  v_status public.ticket_status; v_assignee uuid;
  v_fr timestamptz; v_res timestamptz; v_closed timestamptz; r float8;
begin
  if not public.has_role(auth.uid(), 'admin') then raise exception 'Apenas administradores'; end if;
  if p_count < 1 or p_count > 200 then raise exception 'p_count inválido'; end if;
  select array_agg(id) into v_req from public.profiles;
  select array_agg(user_id) into v_staff from public.user_roles where role in ('admin','agent');
  select array_agg(id) into v_cat from public.categories where active;

  for i in 1..p_count loop
    v_prio := v_prios[1 + floor(random() * array_length(v_prios,1))::int];
    select * into v_sla from public.sla_policies where priority = v_prio;
    r := random();
    v_status := (case when r < 0.55 then 'closed' when r < 0.75 then 'resolved'
                      when r < 0.85 then 'in_progress' when r < 0.93 then 'open'
                      else 'waiting_customer' end)::public.ticket_status;
    if v_status in ('resolved','closed') then
      v_created := now() - make_interval(secs => random() * 60 * 86400);
    else
      v_created := now() - make_interval(secs => random() * 5 * 86400);
    end if;
    v_assignee := case when v_status = 'open' then null
                       else v_staff[1 + floor(random() * array_length(v_staff,1))::int] end;
    v_fr := case when v_status = 'open' then null
                 else least(now(), v_created + make_interval(mins => floor(v_sla.first_response_minutes *
(0.2 + random() * 1.3))::int)) end;
    v_res := case when v_status in ('resolved','closed')
                  then least(now(), greatest(v_fr, v_created + make_interval(mins =>
```

```
floor(v_sla.resolution_minutes * (0.3 + random() * 1.2))::int)))
                else null end;
   v_closed := case when v_status = 'closed' then least(now(), v_res + interval '1 day') else null end;

   insert into public.tickets
    (title, description, category_id, priority, status, requester_id, assignee_id,
     created_at, updated_at, first_response_at, resolved_at, closed_at, is_demo)
   values
    (v_titles[1 + floor(random() * array_length(v_titles,1))::int],
     'Chamado de demonstração gerado automaticamente com dados sintéticos.',
     v_cat[1 + floor(random() * array_length(v_cat,1))::int], v_prio, v_status,
     v_req[1 + floor(random() * array_length(v_req,1))::int], v_assignee,
     v_created, coalesce(v_closed, v_res, v_fr, v_created), v_fr, v_res, v_closed, true);
  end loop;
  return p_count;
end $$;
```

PARTE B — Base de conhecimento (/kb e /kb/:id), todos os usuários logados
- Adicione "Base de conhecimento" à sidebar para todos.
- /kb: campo de busca (debounce 400 ms; busca só com ≥ 2 caracteres) usando supabase .textSearch('search_vector', q, { type: 'websearch', config: 'portuguese' }); filtro por categoria; SEMPRE .eq('status','published') (inclusive para staff). Sem busca: ordene por updated_at desc. Cards com título, categoria e trecho de 140 caracteres do corpo (texto simples). Estado vazio: "Nenhum artigo encontrado".
- /kb/:id: renderiza body_md com react-markdown (sem HTML bruto), título, categoria, data de atualização.
- Texto fixo pequeno abaixo da busca: "A busca é sensível a acentos e exige a palavra completa."

PARTE C — Gerenciar artigos (/kb/manage, SOMENTE staff; link "Gerenciar artigos" na sidebar só para staff)
- Tabela com título, categoria, status (Rascunho/Publicado), atualizado em. Criar/editar em formulário: título (5–150), categoria (opcional), status, body_md (mín. 20) em textarea com alternância Editar/Pré-visualizar. author_id = user.id na criação. Excluir com AlertDialog de confirmação.

PARTE D — Sugestão ao abrir chamado (/tickets/new)
- Enquanto o título tiver ≥ 4 caracteres (debounce 400 ms), busque até 3 artigos publicados via textSearch (mesma config) e mostre card "Artigos que podem ajudar" com links que abrem em nova aba. Não bloqueia o envio. Sem resultado: não mostrar o card.

PARTE E — Administração (/admin, SOMENTE role admin; link "Administração" na sidebar só para admin), com 4 abas:
1. Categorias: listar, adicionar, renomear (nome único, 2–60 caracteres), ativar/desativar. SEM

exclusão.
2. SLA: editar first_response_minutes e resolution_minutes das 4 prioridades (inteiros > 0; resolução ≥ 1ª resposta). Texto fixo: "Alterações valem para novos chamados e para mudanças de prioridade em chamados ainda não resolvidos. SLA corre em horário corrido, sem pausa e sem horário comercial."
3. Usuários: listar profiles com nome, e-mail e select de papel (Solicitante/Agente/Administrador) gravando em user_roles. O select da própria conta do admin fica desabilitado.
4. Demonstração: campo "Quantidade" (1–200, padrão 60) + botão "Gerar chamados de demonstração" (supabase.rpc('seed_demo_tickets', { p_count })); botão "Remover dados de demonstração" (delete em tickets com is_demo = true, com AlertDialog). Aviso fixo: "Os chamados usam as contas existentes como solicitantes. Crie 3–4 contas de teste antes de gerar, ou a taxa de reincidência ficará artificialmente alta."

FEITO QUANDO
- Rotas /kb/manage e /admin inacessíveis a quem não tem o papel (redirecionam para /tickets); a proteção real é o RLS.
- Busca da KB retorna artigos do seed por palavra completa (ex.: "senha").
- Após gerar demo, /dashboard mostra números e gráficos; remover demo zera apenas chamados com is_demo = true.
- Ao final, faça uma revisão por leitura do código e das policies (NÃO afirme que testou no navegador se não testou) e responda SIM/NÃO com 1 linha de evidência para cada item:
  1. Requester A consegue ler chamado de requester B?
  2. Requester consegue ler comentário interno?
  3. Requester consegue alterar status/prioridade/responsável por UPDATE direto?
  4. Algum cálculo de SLA ou métrica é feito no cliente?
  5. Existe qualquer dado mockado no código?
  6. Admin consegue rebaixar a própria conta?

  Respostas esperadas: NÃO, NÃO, NÃO, NÃO, NÃO, NÃO. Se alguma divergir, corrija e informe.

04

29/10

Testar e verificar a segurança

6 / 6

Teste toda lógica gerada por IA. Valide entradas e trate cenários de erro.

- ☑ Testei o fluxo principal com dados fictícios.
- ☑ Testei campos vazios, dados inválidos e entradas inesperadas.
- ☑ Verifiquei mensagens de erro e falhas de integração.
- ☑ Conferi permissões de acesso e proteção de credenciais.
- ☑ Corrigi problemas e repeti os testes.
- ☑ Comparei os resultados com os critérios de sucesso.

Casos de teste, resultados, falhas e correções

Testes manuais com 4 contas fictícias (administrador, técnico, solicitante A e solicitante B) e dados sintéticos. Cada caso foi comparado com os critérios de sucesso C1 a C7 definidos na etapa 1.

F1. O primeiro administrador não pode ser criado pela interface.
Comportamento: todo cadastro entra como Solicitante, e a opção “Administração” não aparece na barra lateral.
Causa: decisão de segurança do projeto, para que ninguém se torne administrador sozinho pelo cadastro.
Correção aplicada: papel alterado manualmente na tabela user_roles, no banco do Lovable Cloud.

F2. Créditos do Lovable esgotados durante o desenvolvimento.
Comportamento: não foi possível solicitar novas correções por prompt.
Causa: limite de créditos do plano.
Correção aplicada: ajustes de dados feitos diretamente nas tabelas; defeitos de código, se houver, ficam registrados como limitações conhecidas.

05

Consolidação final

Documentar a solução

5 / 5

Consolide os registros do projeto e as instruções para utilização. Prazo conforme orientação docente.

- ☑ Descrevi o problema, a solução e a plataforma utilizada.
- ☑ Registrei como configurar e utilizar a solução.
- ☑ Incluí evidências dos testes e das melhorias.
- ☑ Registrei limitações, referências e uso de IA.
- ☑ Revisei os registros antes de imprimir ou salvar em PDF.

Link do projeto, evidências, limitações e próximos passos

Problema, solução e plataforma: ver etapas 1 e 2. A solução é um mini service desk web construído no Lovable com Lovable Cloud.

Como configurar e usar: (1) criar conta em /login, na aba “Criar conta”; (2) o primeiro administrador é promovido por edição direta do papel na tabela user_roles; (3) o administrador define o papel Agente para os técnicos em /admin → Usuários; (4) o solicitante abre chamados em “Novo chamado”; (5) o técnico atende em “Chamados”, com notas internas e mudança de status; (6) o administrador acompanha o /dashboard e pode gerar dados de demonstração em /admin → Demonstração.

Evidências: https://drive.google.com/drive/folders/1AG_TspEHCtn1YpErZLB1C2XGQt3bMock?usp=sharing

Limitações: SLA em horário corrido, sem pausa quando o chamado aguarda o solicitante e sem horário comercial. Reincidência é uma heurística por solicitante e categoria em 30 dias, que não identifica causa raiz. A busca exige a palavra completa e é sensível a acentos. Sem recuperação de senha, notificações por e-mail nem anexos. Fila única compartilhada, sem roteamento por especialidade.

Próximos passos: fila por especialidade do técnico, pausa de SLA, recuperação de senha e notificações por e-mail.

Uso de IA: o código foi gerado pelo Lovable a partir de prompts escritos com apoio do Claude. As decisões de arquitetura, segurança e escopo foram revisadas e escolhidas por mim.

Referências: documentação do Lovable e do Supabase (Row Level Security e funções Postgres).

Link do projeto: https://pixel-perfect-playground-250.lovable.app

Etapas do desenvolvimento · Checklist da equipe · As datas seguem o cronograma informado; a documentação não possui prazo definido neste roteiro.

```
```